TEAM WALKTHROUGH · v3.1.0

MS Word Parser

Forensic analysis of Microsoft Word documents — at the scale of an entire evidence collection.

`.docx` `.docm` `.dotx` `.dotm`

Jacques Boucher and Corey Forman

github.com/jjrboucher/MS-Word-Parser

WHY IT MATTERS

A Word file is a ZIP full of evidence

Every .docx is a compressed package of XML parts. Beyond the visible text, those parts record who created and edited the file, when, in how many sessions, and who commented on it.

Opening the file in Word shows almost none of this — and saving it changes the evidence. MS Word Parser reads the package directly, without altering it, across thousands of files at once.

```
report.docx (ZIP package)
├─ docProps/core.xml ← author, dates
├─ docProps/app.xml ← editing time, app
├─ word/document.xml ← text, RSIDs
├─ word/settings.xml ← RSID Root
├─ word/comments.xml ← comments
├─ word/people.xml ← participants
└─ customXml/item1.xml …
```

HOW IT WORKS

Point it at documents, get structured evidence

INPUT

Choose documents

- Individual files
- A folder, optionally with all subfolders
- A text list of full paths (“ingest”)

PROCESS

Parse in parallel

- Triage or Full mode
- Optional hashing and timeline
- Uses every CPU core; recovers parts from damaged files

OUTPUT

Review results

- Excel workbook (.xlsx)
- SQLite database (.db)
- A log of every step

TWO PROCESSING MODES

Start wide with Triage, then go deep with Full

START HERE

Triage

A fast first pass across thousands of documents to spot what deserves a closer look.

- Document Summary
- Metadata
- Comments
- Excel Tips

DEEP DIVE

Full

Everything in Triage, plus every heavy artifact. Run it on your shortlist — or on big sets with SQLite output.

- RSIDs
- Archive Files
- Track Changes
- Comment threads & reactions
- People
- Protection
- Range Permissions
- Rights Management
- Headers & Footers
- Footnotes & Endnotes
- Custom & Item XML
- Ink

WHAT IT REVEALS

Six kinds of evidence, one run

Authorship & history

Who wrote it, and when?

Author, last editor, created / modified / printed dates, revisions, editing time, template.

Editing sessions

Which documents are related?

RSID Root links files from a common original; rsidR values tie text to save sessions.

Collaboration

Who else touched it?

Comments, replies, resolved status, reactions, people, tracked changes and ink.

Protection & security

Was access restricted?

Encryption, Rights Management validity window, editing restrictions and permitted editors.

The ZIP container

What does the package show?

Every internal part, its size, timestamp and ZIP attributes; hints of non-Word origin.

Document structure

What is inside the body?

Tag counts, document IDs, hyperlinks, headers, footers, notes, proofing state.

SPOTLIGHT · RSIDs

Revision Save IDs link documents together

RSID Root

Set when a document is first created and carried into every copy. Filter on it to find all files derived from the same original.

rsidR

A new random value is generated for each editing session, and text typed in that session is tagged with it — showing what was written when.

Shared rsidRs

rsidR values inherited from a user's Normal.dotm template have a count of 0 in the document. When the same values appear in documents with different RSID Roots, the same Windows user likely created them.

The Excel Tips sheet gives step-by-step pivot-table recipes for each of these checks.

OUTPUTS

Excel for review, SQLite for scale

Excel workbook

- One worksheet per artifact
- Built-in Excel Tips sheet
- Visual Timeline chart
- Large sheets split at 1M rows

SQLite database

- One indexed table per artifact
- Views: Aggregated Comments, Timeline
- 27 ready-made queries on GitHub
- Faster and smaller for Full runs

Optional add-ons

- **Hash:** MD5 of each file and every internal part — integrity and duplicates
- **Timeline:** all timestamps from every artifact in one chronological view

RECOMMENDED WORKFLOW

Triage, shortlist, then go deep

1

Triage all

Run Triage across the full collection.

2

Filter

Sort and filter summary and metadata to find documents of interest.

3

List

Save their full paths to a text file, one per line.

4

Full run

Re-run in Full mode using Ingest and a compiled list of file paths. For large datasets, choose SQLite output.

15K+ files parsed in Full mode… **3m 30s** …in a single run **14M+** RSID rows in one SQLite table

Processing time varies with the capabilities of the computer running it.

GETTING STARTED

Install once, run from the GUI or the command line

Graphical User Interface (GUI)

```
# install with the GUI, then launch it
python3 -m pip install "ms-word-parser[gui]"
parse-docx-gui
```

Launch with `parse-docx-gui` (`parse-docx -g` also works). Pick options, an output folder and files, then Process. Live counts of files, errors and remaining; Stop and Reset at any time.

Command line

```
# triage a folder tree to Excel
parse-docx -tre --dir D:\Evidence -o C:\Out
# full, hashed, timeline, to Excel + SQLite
parse-docx -fesHT --ingest list.txt -o C:\Out
```

Flags without values can be combined: `-fresHT` = `-f -r -e -s -H -T`. Press Space for status, Q to quit. Requires Python 3.9+.

GOOD TO KNOW

Handle the evidence with care

Work on copies

Re-saving a document changes its modified date and last author.

Stored counts can be stale

Page, line and character counts come from the file's own metadata and may be out of date.

Nothing is silently lost

Files that can't be parsed are listed with the reason in the Errors sheet and the log.

Source, research papers and SQL queries: `github.com/jjrboucher/MS-Word-Parser`